

P1378R0

std::string_literal

Published Proposal, 2018-11-26

Author:

[JeanHeyd Meneide](#)

Audience:

SG16

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1378.html

Abstract

This paper proposes a new type `std::basic_string_literal<CharType, N>` that represents and captures only string literals.

Table of Contents

1 Revision History

1.1 Revision 0 - November 26th, 2018

2 Motivation

3 Design

3.1 Safe to convert to NTS

3.2 Backwards Compatibility

3.3 Conversion Rankings

4 Proposed Wording

4.1 Feature Test Macro

4.2 Intent

4.3 Proposed Wording

5 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0 - November 26th, 2018

— Initial release.

§ 2. Motivation

This sort of feature has seen many previous iterations, albeit their motivations were only tangentially related. [\[p0424\]](#) and [\[p0732\]](#) all attempted to create a form of this for various different reasons, from better UDL handling to a proper string type with a fixed backing storage for non-type template parameters. Early forms of this proposal, such as [\[p0259\]](#), also focused on making string-like behaviors available for `constexpr` programming, but were superseded by the previous proposal. Ongoing work has been dedicated to making `std::string` fully `constexpr` to allow for its usage in more complicated syntaxes.

It is clear that `std::string` and `std::string_view` are going to be and already are fully `constexpr`, respectively. This still does not solve the inherent problem that has been run into recently, which is knowing that an argument provided is indeed a string literal. In particular, [\[p1040\]](#) was discussed in the San Diego 2018 ISO C++ Standards Meeting. The primary problem identified was with that of tooling: because a resource identifier could be computed at `constexpr` time and passed as the argument to `std::embed`, then having a proper association between a `std::embed` resource and the object file result is hard to communicate to the user or build system without performing full semantic analysis. Many suggestions were given, but it became apparently that all of the solutions relied on the crux of one common realization: the string for embedded a file can be computed at `constexpr` time, which makes it impossible to extract resource identifier dependency information pre-Phase 7 of compilation.

Since, full semantic analysis must be run before the compiler can be sure of the value passed to `std::embed`, it becomes impossible to list the file as an explicit dependency in the graphs exported by compiler options such as `-MMD`, which only have to perform preprocessing and some basic amount of lexical analysis. It would be better if we could properly express simple dependencies that can participate in the build system without explicitly listing it by ensuring that the value passed to `std::embed` is a string literal.

More broadly, the goal of this type is to solve 2 key use cases. The first is that some functions are increasingly interested in the source of some string data, especially when it comes to strings. Functions that take `const CharType[N]` are inherently losing information from the user: usages of string literals as arrays are lossy transformations that deprive interested parties in necessary source information. For example, is the array backed by compiler-created storage, or did the user create one themselves? Is it null-terminated, or not? We have absolutely no guarantees right now, and that is frustrating to most programmers who care.

Secondly, we care if our tools are able to know the value without having to perform full semantic analysis. A `constexpr std::string_view` or `std::string` do not buy us this guarantee: they can be fully composed at semantic analysis time when the `constexpr`-evaluator runs, generating things that the compiler cannot track until during/after Phase 7 of compilation. This is too late for tools to know the value without significantly slowing down dependency graph generation and other useful compiler services to the build system.

We propose a type that can only be constructed with a non-empty value by the compiler for all the string literal types (`char`, `wchar_t`, `char16_t`, `char32_t`, and `char8_t`). Objects of this type can be captured by application and library developers as the type `std::basic_string_literal<CharType, N>`.

§ 3. Design

This type can implicitly decay to a `const` lvalue reference to an array of `const CharType`. It can be default-constructed and will represent an empty null-terminated (byte) string (NT(B)S), which will be a `const CharType[1]`. As a NTS, the size of the underlying array will be guaranteed to be 1 or more and `*std::end("")` will always be valid and equal to `'\0'` as it is today. The result of a `"asdf"` or similar string literal will always be a `std::basic_string_literal<CharType, N>`:

```

#include <type_traits>
#include <string_literal>

int main () {
    auto x = "woof";
    const auto& x_ref = "purr";
    static_assert(std::is_same_v<std::string_literal<5>, decltype(x)>);
    static_assert(std::is_same_v<std::basic_string_literal<char, 5>, decltype(x_ref)>);
    static_assert(std::is_same_v<const std::basic_string_literal<5>&, decltype(x_ref)>);

    return 0;
}

```

§ 3.1. Safe to convert to NTS

This type avoids the problems of knowing whether a character array or a `std::basic_string_view<CharType>` is a real NTS. This means that rather than having to run `strlen` on the input functions can take the size and immediately know the string has a null terminator at the end. Running afoul of potentially embedded nulls or running off the end of an user-declared array that forgets to null-terminate its storage need not be a concern. APIs transition from accepting the blunt type which does not preserve any source information (e.g. `const char[20]`) to a type that preserves source information and gives up guarantees (e.g. `basic_string_literal<char, 20>`). We can also declare an API that properly handles string literals without decaying to pointers. This can also provide code size and performance benefits, as demonstrated in [Jason Turner's C++Now 2018 talk on initializer_list with various different string types](#) and their interaction with types like `std::string` vs. `std::string_view` vs. `const char*` vs. `const char(&)[N]`.

Another big problem with using purely `std::basic_string_view<CharType>` and/or `const CharType(&)[N]` is the lack of a guarantee about what memory is being referred to by the time it is received in e.g. a function call. Is it truly read-only memory? Did the user initialize an array on the stack that is backing this type? There are so many questions and there is no way to retrieve the answer properly. With this type, we know for sure that the string literal is stored in constant memory that cannot be modified and is null-terminated.

§ 3.2. Backwards Compatibility

It is imperative that this type does not break the assumptions that come from existing code. Because of the implicit conversion to an array on this type, operations such as indexing, arithmetic from the decay-to-pointer, and other properties of the original array type are preserved:

```
// used to be const char[5],
// now is std::basic_string_literal<char, 5>
// NOTE: CTAD (with p1021, approved in San Diego)
// allows us to leave off char/N specifiers in the type name
std::string_literal x = "bark";
// okay: conversion, decay
const char* first = x;
// okay: conversion, decay, then addition
const char* last = x + 5;
// okay: conversion, then indexing operation
char letter_a = x[1];
// okay: conversion, then indexing operation
auto letter_a = x[1];
```

(To see how the CTAD might work in a pre-p1021 (C++17) world, see the stub example working on [Coliru](#) and [Compiler Explorer](#))

The one place where this might provide a breaking change is for users who take the address of a string literal using `&"arf"`. Because the type has changed, this operation may not do what it is expected of code that may have had to use this operation. Currently, the wording synopsis for `std::basic_string_literal<CharType, N>` below does not provide an `operator&`; chiefly, taking the address of a literal directly is an exceedingly rare use case, even for generic code. The author encourages individuals to voice any concerns they have while this potential breakage is considered. If this is deemed a significant use case for strings, then this paper will add the overloaded `operator&` to assuage those compatibility concerns.

There is also little concern about ABI. The `std::basic_string_literal` type is meant to be binary-compatible with a regular built-in array, the same way `std::array` is. Because the type never existed before, name mangling is only a problem for individuals who took built-in arrays as parameters or returned them as values that they expected to be strings with `decltype(auto)`. It seems incredibly unlikely that an interface which returns an array by reference through use of `decltype(auto)` exists and is in prevalent use with C++ compiled at ABI boundaries.

§ 3.3. Conversion Rankings

One of the biggest problems is that the moment someone looks at an array even the tiniest bit funny, it converts down to a pointer to its first element. This has long caused issues of overload resolution (and more issues of people confusing pointers to represent arrays, though this proposal does not solve that unfortunate association). By making it so the type of all string literals are

`std::basic_string_literal<CharType, N>`, this proposal ensures that users can catch the string literal type before the resulting conversion to `const CharType[N]` and the subsequent conversions to `const CharType*`. Note that, unlike user-defined conversions, built-in conversions are allowed to happen an infinite number of times (as compared to user-defined conversions, of which there may only be one on the way to the final destination type). This means that converting to a built-in array allows the regular pointer conversions to happen naturally afterwards, while providing unambiguous overload resolution:

```
#include <string_literal>

template <size_t N>
void f( const std::string_literal<N>& lit ) {
    // 1
}

template <size_t N>
void f( const char(& arr)[N] ) {
    // 2
}

void f( const char* ptr ) {
    // 3
}

int main () {
    const char arr[2]{};
    const char* ptr = arr;
    f(""); // picks 1, unambiguously
    f(arr); // picks 2, unambiguously
    f(ptr); // picks 3, unambiguously
    return 0;
}
```

§ 4. Proposed Wording

Help for wording (especially core) would be appreciated! All wording is relative to [\[n4762\]](#)-ish (for example, this anticipates `char8_t` changes being applied that were approved in San Diego).

§ 4.1. Feature Test Macro

The desired feature test macro for the language change is `__cpp_impl_string_literal`. The desired feature test macro for the library change is `__cpp_lib_string_literal`.

§ 4.2. Intent

The intent of this wording is to supply the following:

- Create a new type `std::basic_string_literal<CharType, N>`.
 - The type shall implicitly convert to an array of `const CharType`.
 - The type shall work with ranged for loops.
 - The type will copy its underlying storage when copied.
 - The type's interface will be entirely read-only / "morally `const`".
 - The type shall be generated only by the core language (except for when default constructed).
- String literals in C++ are now of this type, and they convert to `const CharType(&)[N]` implicitly.
 - They can still be used to initialize character arrays: [§9.3.2 \(dcl.init.string\)](#) remains unchanged.
- It shall not prohibit implementations from storing the data in constant memory as implementations have always done.
- Supply a feature test macro that indicates the core language will generate such a type, `__cpp_impl_string_literals`.
- Supply a feature test macro for the library that provides the type itself, `__cpp_lib_string_literals`.

§ 4.3. Proposed Wording

Modify §5.13.5 [lex.string], clauses 6, 7, 10, 11, and 12 to change the type:

⁶ After translation phase 6, a string-literal that does not begin with an encoding-prefix is an ordinary string literal. An ordinary string literal has type `"array of n const char"` `std::string_literal<n>` (16.  [support.stringlit]) where `n` is the size of the string as defined below, has static storage duration (6.6.4), and is initialized with the given characters.

⁷ A string-literal that begins with `u8`, such as `u8"asdf"`, is a UTF-8 string literal, also referred to as a `char8_t` string literal. A `char8_t` string literal has type `"array of n const char8_t"` `std::u8string_literal<n>` (16.  [support.stringlit]), where `n` is the size of the string as defined below; each successive element of the object representation (6.7) has the value of the corresponding code unit of the UTF-8 encoding of the string.

¹⁰ A string-literal that begins with `u`, such as `u"asdf"`, is a `char16_t` string literal. A `char16_t` string literal has type `"array of n const char16_t"` `std::u16string_literal<n>` (16.  [support.stringlit]), where `n` is the size of the string as defined below; it is initialized with the given characters. A single c-char may produce more than one `char16_t` character in the form of surrogate pairs.

¹¹ A string-literal that begins with `U`, such as `U"asdf"`, is a `char32_t` string literal. A `char32_t` string literal has type `"array of n const char32_t"` `std::u32string_literal<n>` (16.  [support.stringlit]), where `n` is the size of the string as defined below; it is initialized with the given characters.

¹² A string-literal that begins with `L`, such as `L"asdf"`, is a wide string literal. A wide string literal has type `"array of n const wchar_t"` `std::wstring_literal<n>` (16.  [support.stringlit]), where `n` is the size of the string as defined below; it is initialized with the given characters.

Append to §14.8.1 Predefined macro names [cpp.predefined]'s **Table 16** with one additional entry:

Macro name	Value
<code>_cpp_impl_string_literals</code>	<code>201902L</code>

Append to §16.3.1 General [support.limits.general]'s **Table 35** one additional entry:

Macro name	Value
<code>_cpp_lib_string_literals</code>	<code>201902L</code>

Add an entry to §16.1 General [**support.general**] as follows:

	Subclause	Header(s)
16.❖	<code>String Literals</code>	<code><string_literal></code>

Add a new section §16.❖ [**support.stringlit**]:

16.◆ String Literals

[support.stringlit]

The header `<string_literal>` defines a class template and several support functions related to string literals (5.13.5 [lex.string]). All functions specified in this sub-clause are signal-safe (16.12.4 [support.signal]).


```

namespace std {
    template <class CharType, std::size_t N>
    class basic_string_literal {
    private:
        using storage_type      = CharType[N]; // exposition-only
    public:
        using value_type        = CharType;
        using reference          = const CharType&;
        using const_reference    = const CharType&;
        using size_type          = size_t;

        using iterator           = const CharType*;
        using const_iterator      = const CharType*;

        // 16.0.2, String literal constructor
        constexpr basic_string_literal() noexcept;

        // 16.0.3, String literal conversions
        constexpr operator const storage_type& () const;

        // 16.0.4, String literal access
        constexpr const CharType* data() const noexcept;
        constexpr const CharType* c_str() const noexcept;

        constexpr size_type size() const noexcept;

        constexpr iterator begin() const noexcept;
        constexpr iterator end() const noexcept;
        constexpr const_iterator cbegin() const noexcept;
        constexpr const_iterator cend() const noexcept;

    private:
        storage_type arr; // exposition-only
    };

    // 16.0.5, string literal range access
    template <class CharType, size_t N>
    constexpr const CharType* begin (const basic_string_literal&) noexcept;
    template <class CharType, size_t N>
    constexpr const CharType* end (const basic_string_literal&) noexcept;
    template <class CharType, size_t N>
    constexpr const CharType* cbegin (const basic_string_literal&) noexcept;
    template <class CharType, size_t N>
    constexpr const CharType* cend (const basic_string_literal&) noexcept;

```

```

template <class CharType, size_t N>
constexpr const CharType* cend (const basic_string_literal&) noexcept

template
basic_string_literal( const CharType(&)[N] ) -> basic_string_literal

template
using string_literal = basic_string_literal;

template
using wstring_literal = basic_string_literal;

template
using u8string_literal = basic_string_literal;

template
using ul6string_literal = basic_string_literal;

template
using u32string_literal = basic_string_literal;

}

```

- 1 An object of type `basic_string_literal<CharType, N>` provides access to an array of objects of type `const CharType`. [Note: Copying a `basic_string_literal<CharType, N>` copies the underlying elements. —end note]
- 2 If an explicit specialization or partial specialization of `basic_string_literal` is declared, the program is ill-formed.

16.2 String literal constructors

[stringlit.cons]

```
constexpr basic_string_literal() noexcept;
```

1 *Effects:* Constructs an empty `basic_string_literal` object.

2 *Ensures:* `size() == 0` and `*end() == '\0'`.

16.3 String literal conversions

[stringlit.conv]

```
operator storage_type&() const noexcept;
```

3 *Effects:* returns `arr`.

16.4 String literal access

[stringlit.access]

```
constexpr const CharType* data() const noexcept;
```

1 *Effects:* returns `arr`.

```
constexpr const CharType* c_str() const noexcept;
```

2 *Effects:* returns `arr`.

```
constexpr size_type size() const noexcept;
```

3 *Effects:* returns `N`.

```
constexpr const CharType* begin() const noexcept;
```

4 *Effects:* returns `arr`.

```
constexpr const CharType* end() const noexcept;
```

5 *Effects:* returns `begin() + size()`.

```
constexpr const CharType* cbegin() const noexcept;
```

6 *Effects:* returns `arr`.

```
constexpr const CharType* cend() const noexcept;
```

7 *Effects:* returns `cbegin() + size()`.

16.5 String literal range access

[stringlit.range]

```
template <CharType, size_t N> constexpr const CharType* begin (const  
basic_string_literal<CharType, N>& lit) noexcept;
```

1 *Effects: returns `lit.begin()`.*

```
template <CharType, size_t N> constexpr const CharType* end (const  
basic_string_literal<CharType, N>&) noexcept;
```

2 *Effects: returns `lit.end()`.*

```
template <CharType, size_t N> constexpr const CharType* cbegin (const  
basic_string_literal<CharType, N>&) noexcept;
```

3 *Effects: returns `lit.cbegin()`.*

```
template <CharType, size_t N> constexpr const CharType* cend (const  
basic_string_literal<CharType, N>&) noexcept;
```

4 *Effects: returns `lit.cend()`.*

Modify §20.3.2 [basic.string] to add the following constructor:

```
template <size_t N> basic_string(const basic_string_literal<charT, N>&);
```

Modify §20.3.2.3 [string.cons] to add the following constructor:

```
template <size_t N> basic_string(const basic_string_literal<charT, N>&  
lit);
```

◆ *Effects: behaves the same as if invoking: `basic_string(lit.data(), lit.size())`.*

Modify §20.4.2 [string.view.template] to add the following constructor:

```
template <size_t N> basic_string(const basic_string_literal<charT, N>&);
```

Modify §20.4.2.1 [string.view.cons] to add the following constructor:

```
template <size_t N> basic_string_view(const basic_string_literal<charT,
N>& lit);
```

◆ *Effects: behaves the same as if invoking:* `basic_string_view(lit.data(),
lit.size())`.

§ 5. Acknowledgements

Thanks to Colby Pike (vector-of-bool) for helping to incubate and brew this idea. Thanks to Jason Turner for elaborating in quite a bit of detail the pitfalls of string initialization and the need for a string literal type.

§ References

§ Informative References

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4762 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf). May 7th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf>

[P0259]

Michael Price; Andrew Tomazos. [fixed_string: a compile-time string](https://wg21.link/p0259). 2016. URL: <https://wg21.link/p0259>

[P0424]

Louis Dionne; Hana Dusíková. [String literals as non-type template parameters](https://wg21.link/p0424). November 14th, 2017. URL: <https://wg21.link/p0424>

[P0732]

Jeff Snyder; Louis Dionne. [Class Types in Non-Type Template Parameters](https://wg21.link/p0732). June 6th, 2018. URL: <https://wg21.link/p0732>

[P1040]

JeanHeyd Meneide. [std::embed](https://wg21.link/p1040). October 12th, 2018. URL: <https://wg21.link/p1040>